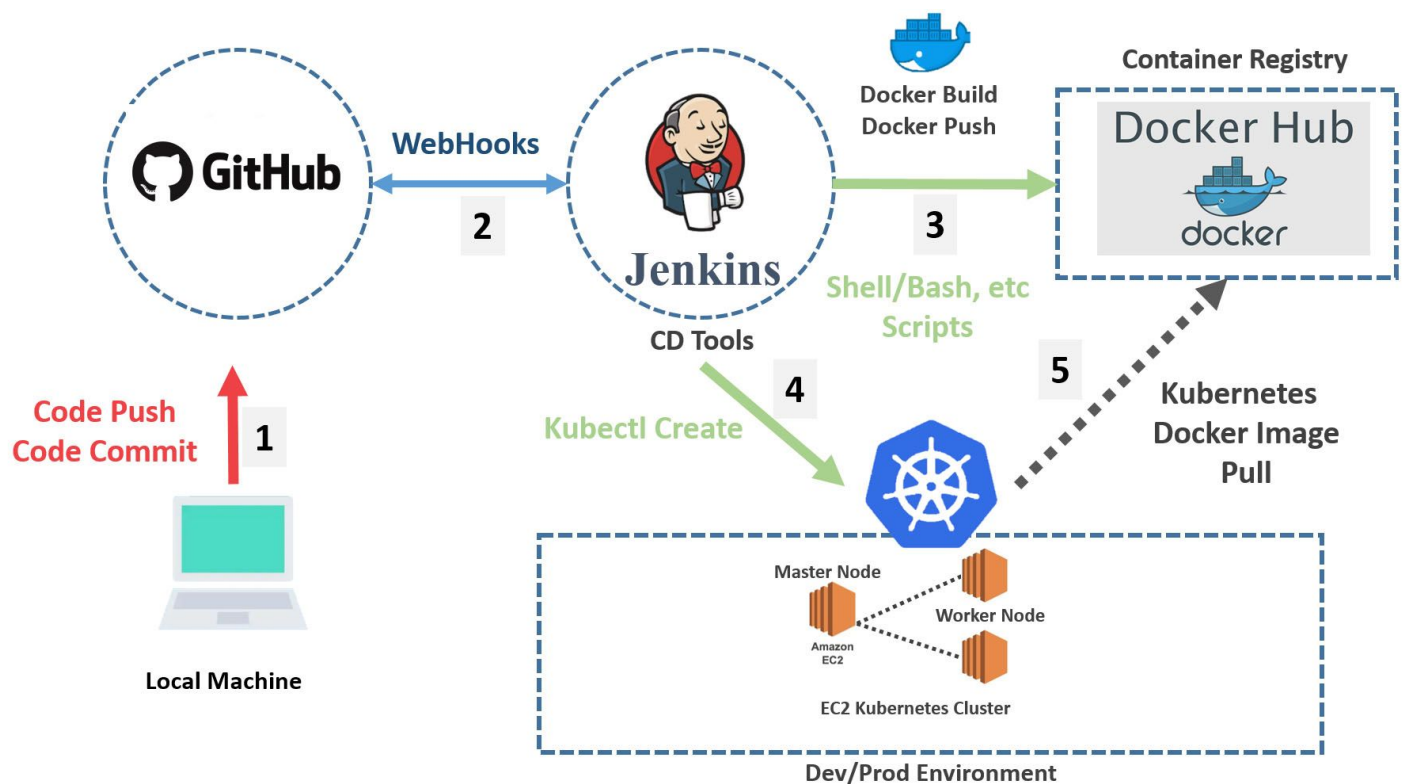


Mini Projet

Jenkins - Docker - Kubernetes - Test

Architecture générale



Installer Jenkins avec Docker

- Créer un dossier `tp_jenkins`.
- Cherchez sur hub.docker.com l'image `blueocean` de `jenkins`.
- Créez dans `tp_jenkins` un dossier `jenkins_simple`.

- Ouvrez `jenkins_simple` avec VSCode. Pour lancer jenkins nous allons utiliser docker compose.
- Créez un fichier `docker-compose.yml` avec à l'intérieur:

```
version: "2"
services:
  jenkins:
    image: <image_blueocean>
    user: root
    ports:
      - "<port_jenkins_standard>"
    volumes:
      ...
```

- Pour le mapping de port choisissez `8080` pour le port hôte (le port http par défaut voir cours sur le réseau). Le port de jenkins est `8080`.
- Créez dans `jenkins_simple` les dossiers `home` et `jenkins_data`
- La section volumes permet de monter des volumes docker :
 - les données de jenkins se retrouveront dans le dossier `jenkins_data` et survivront à la destruction du conteneur. À l'intérieur du conteneur le dossier data est `/var/jenkins`
 - Le dossier `./home` peut également être monté à l'emplacement `/home` pour persister les données de build.
 - Enfin pour que jenkins puisse utiliser Docker il doit pouvoir accéder au socket docker de l'hôte qui permet de contrôler la runtime docker. Il faut pour cela monter `/var/run/docker.sock` au même emplacement (`/var/run/docker.sock`) côté conteneur.
- Après avoir complété le fichier et ajouté les 3 volumes, lancez jenkins avec `docker-compose up -d`.
- Pour vérifier que le conteneur est correctement démarré utilisez la commande `docker-compose logs`
- Quand le logiciel est prêt la commande précédente affiche des triple lignes d'étoiles `*`. Entre les deux est affiché un token du type: `05992d0046434653bd253e85643bae12`. Copiez ce token.
- Visitez l'adresse `http://localhost:8080`. Vous devriez voir une page jenkins s'afficher. Activez le compte administrateur avec le token précédemment récupéré.
- Cliquez sur `Installer les plugins recommandés`
- Créez un utilisateur à votre convenance. Jenkins est prêt.

Créer un premier pipeline

- Cliquez sur créer un nouvel item, sélectionnez le type `pipeline`.
- Dans le vaste formulaire qui s'ouvre remplissez la section nom avec `hello`

- Laissez tout le reste tel quel sauf la section **script** en bas ou il faut coller la description du pipeline:

```
pipeline {
    agent any
    stages {
        stage("Hello") {
            steps {
                echo 'Hello World'
            }
        }
    }
}
```

- Sauvegardez le pipeline. Retournez sur la page d'accueil de Jenkins et lancez votre tâche.
- Cliquez sur le job qui se lance **#1** ou **#2** pour suivre son déroulement puis cliquez sur **Console Output** dans le menu de gauche.
- Vous devriez voir quelque chose comme:

```
Started by user elie
Running in Durability level: MAX_SURVIVABILITY
[Pipeline] Start of Pipeline
[Pipeline] node
Running on docker-slave-41a6ab3a5327 in /home/jenkins/workspace/hello
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Hello)
[Pipeline] echo
Hello World
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

- L'interface douteuse que vous venez de visiter est celle de jenkins traditionnelle. Nous allons maintenant voir BlueOcean qui est plus simple et élégante.
- Cliquez sur **Open Blue Ocean**
- Affichez simplement les logs de notre pipeline précédent. La mise en forme est plus épurée et claire.
- Pour accéder directement à la page d'accueil visitez **<http://localhost:8080/blue>** .
- Cliquez sur le job hello et relancez le. Un nouveau pipeline démarre qui s'exécute en une seconde.

Passons maintenant à un vrai pipeline de test. Pour cela nous devons d'abord avoir une application à tester et un jeu de tests à appliquer. Nous allons comme dans les TP précédents utiliser une application python flask.

Tester une application flask avec Jenkins

- Dans `tp_jenkins` créer un dossier `flask_app`.
- Ouvrez ce dossier avec une nouvelle instance de VSCode.
- Créez à l'intérieur `.gitignore`, `app.py`, `requirements.txt` et `test.py`.

`.gitignore`

```
__pycache__  
*.pyc  
venv
```

`app.py`

```
#!/usr/bin/env python3  
from flask import Flask  
app = Flask(__name__)  
  
@app.route('/')  
@app.route('/hello/')  
def hello_world():  
    return 'Hello World!\n'  
  
@app.route('/hello/<username>') # dynamic route  
def hello_user(username):  
    return 'Hello %s!\n' % username  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0') # open for everyone
```

`requirements.txt`

```
Click==7.0  
Flask==1.0.2  
itsdangerous==1.1.0  
Jinja2==2.10  
MarkupSafe==1.1.0  
Werkzeug==0.14.1  
xmlrunner==1.7.7
```

`test.py`

```
#!/usr/bin/env python3

import unittest
import app

class TestHello(unittest.TestCase):

    def setUp(self):
        app.app.testing = True
        self.app = app.app.test_client()

    def test_hello(self):
        rv = self.app.get('/')
        self.assertEqual(rv.status, '200 OK')
        self.assertEqual(rv.data, b'Hello World!\n')

    def test_hello_hello(self):
        rv = self.app.get('/hello/')
        self.assertEqual(rv.status, '200 OK')
        self.assertEqual(rv.data, b'Hello World!\n')

    def test_hello_name(self):
        name = 'Simon'
        rv = self.app.get(f'/hello/{name}')
        self.assertEqual(rv.status, '200 OK')
        self.assertIn(bytearray(f"{name}", 'utf-8'), rv.data)

if __name__ == '__main__':
    unittest.main()
```

- Pour essayer nos tests sur l'application lancez:

- `virtualenv -p python3 venv`
- `source venv/bin/activate`
- `pip install -r requirements.txt`
- `chmod +x app.py test.py`
- `./test.py --verbose`

résultat:

```
test_hello (__main__.TestHello) ... ok
test_hello_hello (__main__.TestHello) ... ok
test_hello_name (__main__.TestHello) ... ok
```

```
-----
Ran 3 tests in 0.014s
```

OK

Créons maintenant un conteneur docker avec cette application. Précréer une image Docker permet de tester plus facilement les applications dans un contexte d'automatisation CI/CD car l'application est encapsulée et facile à déployer pour Jenkins (ou autre ex gitlab).

- Créez le `Dockerfile` suivant dans `flask_app` :

```
FROM python:3.7

# Ne pas lancer les app en root dans docker
RUN useradd flask
WORKDIR /home/flask

#Ajouter tout le contexte sauf le contenu de .dockerignore
ADD . .

# Installer les déps python, pas besoin de venv car docker
RUN pip install -r requirements.txt
RUN chmod a+x app.py test.py && \
    chown -R flask:flask ./

# Déclarer la config de l'app
ENV FLASK_APP app.py
EXPOSE 5000

# Changer d'user pour lancer l'app
USER flask

CMD ["/app.py"]
```

- Ajoutez également le `.dockerignore` :

```
__pycache__
venv
Dockerfile
```

- Construisez l'image : `docker build -t flask_hello .`
- Lancez la pour tester son fonctionnement: `docker run --rm --name flask_hello -p 5000:5000 flask_hello`
- Pour lancez les test il suffit d'écraser au moment de lancer le conteneur la commande par défaut `./app.py` par la commande `./test.py` :

```
docker run --rm --name flask_hello -p 5000:5000 flask_hello ./test.py --v
```



- Lancez un registry docker en local sur le port 4000 avec `docker run -p 4000:5000 registry` LAISSEZ LE TOURNER.

- tagguez l'image pour la pousser sur ce registry : `docker tag flask_hello localhost:4000/flask_hello`
- poussez la avec `docker push localhost:4000/flask_hello`

Installer Jenkins dans Kubernetes

Nous allons installer Jenkins avec Helm et le chart Jenkins Stable. Cela va permettre d'utiliser des pods k8s comme agents Jenkins. Pour cela :

- Commencez par récupérer le fichier des valeurs de configuration du chart Jenkins stable avec : `wget https://raw.githubusercontent.com/helm/charts/master/stable/jenkins/values.yaml`

Ce fichier contient plein d'options sur comment configurer l'installation de Jenkins et Jenkins lui même:

- sur quel port exposer le service
- quels plugins Jenkins installer
- le nom et mot de passe de l'admin
- etc
- Modifiez: `adminUser` et `adminPassword` pour mettre les vôtres
- Modifiez `serviceType: NodePort`
- Ajoutez `nodePort: 32000` à la ligne juste en dessous de `servicetype`

Maintenant lançons l'installation avec :

- `kubectl create namespace jenkins`
- `kubectl config set-context --current --namespace=jenkins`
- `helm repo add stable https://kubernetes-charts.storage.googleapis.com/`
- `helm repo update`
- `helm install --name-template jenkins -f values.yaml stable/jenkins`
- Chargez la page localhost:32000 pour accéder à Jenkins et utilisez le login configuré.
- Installez le plugin blue ocean dans l'administration de Jenkins

Tester notre application avec un pipeline

Pour tester notre application nous allons créer un `pipeline as code` c'est à dire ici un fichier `Jenkinsfile` à la racine de l'application `flask_app` qui décrit notre test automatique.

`Jenkinsfile`

```
pipeline {
  agent {
    kubernetes {
```

```

        // this label will be the prefix of the generated pod's name
        label 'jenkins-agent-my-app'
    yaml """
apiVersion: v1
kind: Pod
metadata:
  labels:
    component: ci
spec:
  containers:
    - name: python
      image: python:3.7
      command:
        - cat
      tty: true
"""
  }
}

stages {
  stage('Test python') {
    steps {
      container('python') {
        sh "pip install -r requirements.txt"
        sh "python test.py"
      }
    }
  }
}
}
}

```

- Créez un commit pour le code de l'application.
- Créez un nouveau projet github `flask_hello_jenkins` et poussez le code.
- Copiez l'adresse SSH de votre nouveau dépôt (menu `clone` de la page d'accueil).
- Allez dans Blue Ocean et créez un nouveau pipeline de type `git` (pas github ou autre) en collant l'adresse SSH précédent.
- Blue Ocean vous présente une clé SSH et vous propose de l'ajouter à votre dépôt pour l'authentification.
 - Cliquez sur `copy to clipboard`
 - Allez dans github > settings > SSH keys et ajoutez la clé
 - retournez sur la page jenkins et faite `Créer le pipeline`
- A partir d'ici le pipeline démarre
 - d'abord (étape 1) Jenkins s'authentifie en SSH et clone le dépôt du projet automatiquement
 - puis il lit le `Jenkinsfile` et crée les étapes (steps) nécessaires à partir de leur définition

- `agent { kubernetes ... containers: ... - python:3.7.2' }` indique que Jenkins doit utiliser un pod kubernetes basé sur l'image docker python pour exécuter les test
 - l'étape `Test Python` installe les requirements nécessaire et lance simplement le fichier de test précédemment créé
- Observez comment Blue ocean créé des étapes d'une chaîne (le pipeline) et vous permet de consulter les logs détaillés de chaque étape.

Ajouter le déclenchement automatique du pipeline à chaque push.

Ajoutons un trigger Jenkins pour déclencher automatiquement le pipeline dès que le code sur le dépôt change. Une façon simple de faire cela est d'utiliser le trigger `pollSCM` :

- Ajouter en dessous de la ligne `agent` du `Jenkinsfile` les trois lignes suivantes:

```
triggers {
    pollSCM('* * * * *')
}
```

Cet ajout indique à Jenkins de vérifier toute les minute si le dépôt à été mis à jour.

- Créez un commit et poussez le code.
- Pour que le trigger soit pris en compte il faut d'abord relancer le build manuellement : allez sur l'interface et relancez le build. Normalement les tests sont toujours fonctionnels.

Utiliser le Dockerfile pour construire un artefact

Dans le `Jenkinsfile` précédent nous avons demandé à Jenkins de partir de l'image `python:3.7.2` et d'installer les requirements.

Mais nous avons aussi un Dockerfile qui va nous permettre de construire l'image et la pousser automatiquement sur notre registry. Ajoutez le stage build suivant

```
stage('Build image') {
    steps {
        container('docker') {
            sh "docker build -t localhost:4000/pythontest:latest ."
            sh "docker push localhost:4000/pythontest:latest"
        }
    }
}
```

Pour builder l'image le contexte du pipeline doit avoir docker disponible. Pour cela nous allons ajouter un deuxième conteneur et un volume à notre pod jenkins agent. Ajoutez un deuxième conteneur à la liste `spec: containers:` de la configuration de l'agent kubernetes comme suit:

```
- name: docker
  image: docker
  command:
    - cat
  tty: true
  volumeMounts:
    - mountPath: /var/run/docker.sock
      name: docker-sock
volumes:
- name: docker-sock
  hostPath:
    path: /var/run/docker.sock
```

- Commitez, poussez cette version et relancez le pipeline.
- Observez les logs en particulier la partie build de l'image : Jenkins utilise docker pour relancer le build

Déployer l'application dans Kubernetes

Nous allons enfin ajouter un stage Deploy pour lancer notre application dans le cluster et pouvoir la tester.

créez un dossier `kubernetes` avec à l'intérieur deux fichiers:

- `deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pythontest
  labels:
    app: pythontest
spec:
  selector:
    matchLabels:
      app: pythontest
  strategy:
    type: Recreate
  replicas: 1
  template:
    metadata:
      labels:
        app: pythontest
    spec:
      containers:
```

- `image: localhost:4000/pythontest:latest`
`name: pythontest`
`ports:`
 - `containerPort: 5000`
`name: microport`

- `service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: pythontest
  labels:
    app: pythontest
spec:
  ports:
    - port: 5000
      nodePort: 31000
  selector:
    app: pythontest
  type: NodePort
```

Ajoutez un stage Deploy au pipeline Jenkinsfile comme suit:

```
stage('Deploy') {
  steps {
    container('kubectl') {
      sh "kubectl apply -f ./kubernetes/deployment.yaml"
      sh "kubectl apply -f ./kubernetes/service.yaml"
    }
  }
}
```

Pour exécuter ces commande il nous faut kubectl dans le pod. Ajoutons pour cela un conteneur kubectl dans le pod à la suite du conteneur docker précédemment ajouté et avant la section `volumes:` :

- `name: kubectl`
`image: lachlanevenson/k8s-kubectl:v1.17.2` # use a version that match
`command:`
 - `cat``tty: true`



- Poussez ces modification et lancez un build à nouveau.

Ajoutez une branche feature et du TDD

Nous allons maintenant ajouter un test pour une fonctionnalité hypothétique non encore existante. Le test va donc échouer. Ecrire le test à l'avance fait partie d'une méthode appelée TDD ou développement dirigé par les tests.

- Normalement on ne fait pas ça (ajouter un test en échec) dans la branche master mais dans une branche feature (sinon on casse la branche stable).
Basculez sur une branche `feature1` avec `git checkout -b <branche>`
- Ajoutez le code suivant à `test.py` après `test_hello_name` :

```
def test_new_route(self):  
    rv = self.app.get(f'/feature/{name}')
```

`self.assertEqual(rv.status, '200 OK')`

- Committez et pousser votre code.
- Rendez vous sur l'interface de Blue Ocean. Au bout de 2 minutes, vous pourrez observer un nouveau pipeline en échec.
 - Le pipeline de test s'est lancé automatiquement sur la nouvelle branche `feature1`.
 - La sortie indique: `Ran 4 tests in 0.015s FAILED (errors=1)`
- Ajoutez une route python flask dans `app.py` pour que le test fonctionne:
 - indice1: dupliquez la route `/hello/<username>` et sa fonction
 - indice2: changez la route en avec `/feature`.
- Vous pouvez tester votre ajout en local en relançant : `./test.py`
- Une fois que le test est corrigé, poussez votre "feature" sur framagit et observez le pipeline de test.
- Normalement, le pipeline il est toujours en échec. Expliquez ce qui s'est passé.
- Corrigez le problème dans la branche feature en remettant l'agent comme précédemment. Mettez à jours le projet sur framagit.
Le pipeline devrait maintenant passer à nouveau.